

# PoRE Lab4 实验报告

姓名：徐锦云

学号：23307130447

耗时：12小时

## PoRE Lab4 实验报告

### Task 1: 修改数据字节 (20%)

编译和运行命令、objdump查看修改前文件汇编、并指出需要修改的位置

二进制文件编辑器中修改内容截图

运行修改后二进制文件截图

### Task 2: 修改指令字节 (20%)

编译和运行命令、objdump查看修改前文件汇编、并指出需要修改的位置

二进制文件编辑器中修改内容截图

运行修改后二进制文件截图

### Task 3: 修改PLT/GOT表劫持libc函数 (60%)

编译和运行命令、objdump查看修改前文件汇编、并指出需要分析的部分

调试以输出完整字符串

PLT/GOT表偏移计算过程

二进制文件编辑器中修改内容截图

运行修改后二进制文件截图

## Task 1: 修改数据字节 (20%)

### 编译和运行命令、objdump查看修改前文件汇编、并指出需要修改的位置

```
aarch64-linux-gnu-gcc -O0 -g -o task12 Lab4_Task_1_2_23307130447.c
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task12 test
```

编译运行。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [8:23:13] C:100
● $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task12 test
You failed
```

现在用objdump查看：

```
aarch64-linux-gnu-objdump -d ./task12
```

我们关注main函数部分：

```

000000000000009cc <main>:
 9cc: d10283ff      sub     sp, sp, #0xa0
 9d0: a9097bfd      stp     x29, x30, [sp, #144]
 9d4: 910243fd      add     x29, sp, #0x90
 9d8: b9000fe0      str     w0, [sp, #12]
 9dc: f90003e1      str     x1, [sp]
 9e0: f00000e0      adrp    x0, 1f000 <__FRAME_END__+0x1e434>
 9e4: f947f400      ldr     x0, [x0, #4072]
 9e8: f9400001      ldr     x1, [x0]
 9ec: f90047e1      str     x1, [sp, #136]
 9f0: d2800001      mov     x1, #0x0                                // #0
 9f4: 97ffffc9      bl      918 <dummy_func>
 9f8: 910043e0      add     x0, sp, #0x10
 9fc: d2800ea1      mov     x1, #0x75                                // #117
a00: 97ffffc8      bl      920 <rand_str>
a04: f94003e0      ldr     x0, [sp]
a08: 91002000      add     x0, x0, #0x8
a0c: f9400001      ldr     x1, [x0]
a10: 910043e0      add     x0, sp, #0x10
a14: d2800e82      mov     x2, #0x74                                // #116
a18: 97ffff6a      bl      7c0 <memcmp@plt>
a1c: 7100001f      cmp     w0, #0x0
a20: 54000081      b.ne    a30 <main+0x64> // b.any
a24: 910043e0      add     x0, sp, #0x10
a28: 97ffff62      bl      7b0 <puts@plt>
a2c: 14000004      b       a3c <main+0x70>
a30: 90000000      adrp    x0, 0 <__abi_tag-0x278>
a34: 912a4000      add     x0, x0, #0xa90
a38: 97ffff5e      bl      7b0 <puts@plt>
a3c: 52800000      mov     w0, #0x0                                // #0
a40: 2a0003e1      mov     w1, w0
a44: f00000e0      adrp    x0, 1f000 <__FRAME_END__+0x1e434>
a48: f947f400      ldr     x0, [x0, #4072]
a4c: f94047e3      ldr     x3, [sp, #136]
a50: f9400002      ldr     x2, [x0]
a54: eb020063      subs    x3, x3, x2
a58: d2800002      mov     x2, #0x0                                // #0
a5c: 54000040      b.eq    a64 <main+0x98> // b.none
a60: 97ffff48      bl      780 <__stack_chk_fail@plt>
a64: 2a0103e0      mov     w0, w1
a68: a9497bfd      ldp     x29, x30, [sp, #144]
a6c: 910283ff      add     sp, sp, #0xa0
a70: d65f03c0      ret

```

我们根据原来的C代码来理解：

```
int main(int argc, char **argv) {
    char flag[0x75];
    dummy_func();
    rand_str(flag, 0x75);

    if (!memcmp(flag, argv[1], 0x74))
        puts(flag);
    else
        puts("You failed");

    return 0;
}
```

memcmp 函数的功能：传入两个指针 str1、一个指针指向的内存块的前n个字节。如果 str1 等于 str2，以及一个无符号整数n。比较两 str2，则返回0；否则不为0。

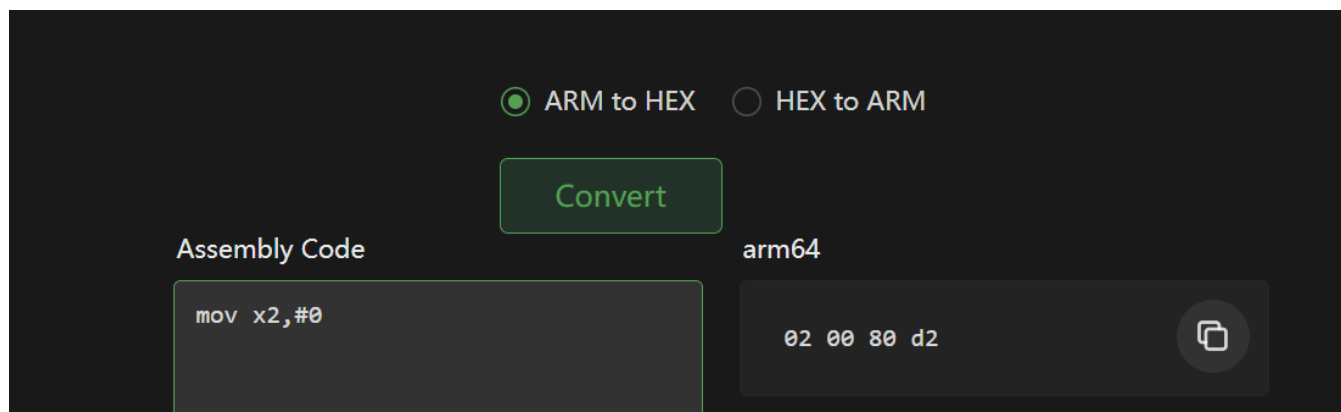
我们现在的结果是fail，因此如果从修改数据的角度看，我们应该把比较的字节数改成0。

这个对应的就是：

```
a14: d2800e82      mov     x2, #0x74      // #116
```

我们需要把这个改成 `mov x2, #0`

## 二进制文件编辑器中修改内容截图



我们需要在地址a14处修改，并且输入小端法：

```
02 00 80 d2
```

下载Hex Editor插件，然后修改。

原先的截图：

```
00000A10 E0 43 00 91 82 0E 80 D2 6A FF FF 97 1F 00 00 71
```

修改之后的截图：

```
00000A10 E0 43 00 91 02 00 80 D2 6A FF FF 97 1F 00 00 71
```

另：为了把insert改成overwrite：

- `Ctrl + Shift + P` 打开命令面板
- 输入并执行 `Toggle Overwrite Mode` (切换覆盖模式)

这个情况是可以直接修改的，但是如果不行也可以在终端修改：

```
printf '\x80' | dd of=task2 bs=1 seek=$((0xA20)) conv=notrunc status=none
```

这个的意思是把task2文件地址A20处一个字节替换为80，它具有同样的效力。

## 运行修改后二进制文件截图

文件另存为task1，执行：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task1 test
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [8:54:42]
• $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task1 test
buppjkssftpudotbrlyhcspegvomhnmkzbzrnltnapsqiwkvwrxogbwdrkeuymyvflosgbvwxwlfsvdqochwedzxmlcihqi
cvutncwkaswhmtkdknkg
```

## Task 2：修改指令字节（20%）

### 编译和运行命令、objdump查看修改前文件汇编、并指出需要修改的位置

```
aarch64-linux-gnu-gcc -O0 -g -o task12 Lab4_Task_1_2_23307130447.c
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task12 test
```

编译运行。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [8:23:13] C:100
• $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task12 test
You failed
```

在汇编代码中，如果 `memcmp` 返回值不为0，则跳转到其他位置执行 `puts("You failed")`，否则 顺序执行 `puts(flag)`。所以需要修改的是：

```
a1c: 7100001f      cmp     w0, #0x0
a20: 54000081      b.ne   a30 <main+0x64> // b.any
```

既然现在memcmp得到的值不是0（也就是两个字符串不相同），那么就让条件反转，就能正确输出了。

我们需要做的是把地址a20处的b.ne改成b.eq

## 二进制文件编辑器中修改内容截图

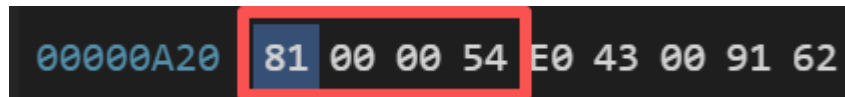
我们需要的指令：

```
b.eq a30 <main+0x64>
```

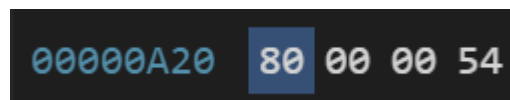
对应的机器码：54 00 00 80，转换成小端法：

```
80 00 00 54
```

原先的截图：



修改后的截图：



## 运行修改后二进制文件截图

将文件另存为task2，然后执行：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task2 test
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [9:26:56]
• $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task2 test
bupjksftputdotbrlyhcspegvomhnmkzbzrnltnapsqiwkvwrxogbwdrkeuuyvflosgbvxwlfsvdqochwedzxmlicihqicvutncwka
swhmtkdknkg
```

## Task 3：修改PLT/GOT表劫持libc函数（60%）

### 编译和运行命令、objdump查看修改前文件汇编、并指出需要分析的部分

```
aarch64-linux-gnu-gcc -O0 -g -o task3 Lab4_Task_3_23307130447.c
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3
```

编译运行。

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [9:29:47]
• $ aarch64-linux-gnu-gcc -O0 -g -o task3 Lab4_Task_3_23307130447.c

# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [9:31:11]
• $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3
touch /tmp/Lab4_task3_not_you_need
fail
```

现在用objdump查看：

```
aarch64-linux-gnu-objdump -d ./task3
```

我们关注main函数部分：

```
0000000000000b04 <main>:
b04: d10183ff      sub     sp, sp, #0x60
b08: a9057bfd      stp     x29, x30, [sp, #80]
b0c: 910143fd      add     x29, sp, #0x50
b10: f00000e0      adrp    x0, 1f000 <__FRAME_END__+0x1e240>
b14: f947f400      ldr     x0, [x0, #4072]
b18: f9400001      ldr     x1, [x0]
b1c: f90027e1      str     x1, [sp, #72]
b20: d2800001      mov     x1, #0x0                                // #0
b24: 90000000      adrp    x0, 0 <__abi_tag-0x278>
b28: 91310001      add     x1, x0, #0xc40
b2c: 910023e0      add     x0, sp, #0x8
b30: ad400420      ldp     q0, q1, [x1]
b34: ad000400      stp     q0, q1, [x0]
b38: b841f021      ldur    w1, [x1, #31]
b3c: b801f001      stur    w1, [x0, #31]
b40: 9100afe0      add     x0, sp, #0x2b
b44: 4f000400      movi    v0.4s, #0x0
b48: 3d800000      str     q0, [x0]
b4c: 3c80d000      stur    q0, [x0, #13]
b50: 97ffffa2      bl      9d8 <rand_func>
b54: 910023e0      add     x0, sp, #0x8
b58: 97ffff56      bl      8b0 <puts@plt>
b5c: 90000000      adrp    x0, 0 <__abi_tag-0x278>
b60: 91302000      add     x0, x0, #0xc08
b64: 97ffff37      bl      840 <system@plt>
b68: 97ffffba      bl      a50 <file_checker>
b6c: 52800000      mov     w0, #0x0                                // #0
b70: 2a0003e1      mov     w1, w0
b74: f00000e0      adrp    x0, 1f000 <__FRAME_END__+0x1e240>
b78: f947f400      ldr     x0, [x0, #4072]
b7c: f94027e3      ldr     x3, [sp, #72]
b80: f9400002      ldr     x2, [x0]
b84: eb020063      subs    x3, x3, x2
b88: d2800002      mov     x2, #0x0                                // #0
b8c: 54000040      b.eq    b94 <main+0x90> // b.none
b90: 97ffff30      bl      850 <__stack_chk_fail@plt>
b94: 2a0103e0      mov     w0, w1
b98: a9457bfd      ldp     x29, x30, [sp, #80]
b9c: 910183ff      add     sp, sp, #0x60
ba0: d65f03c0      ret
```

我们的重点在于：

```
b24: 90000000      adrp    x0, 0 <__abi_tag-0x278>
b28: 91310001      add     x1, x0, #0xc40
b2c: 910023e0      add     x0, sp, #0x8
b30: ad400420      ldp     q0, q1, [x1]
b34: ad000400      stp     q0, q1, [x0]
```

```

b38:  b841f021    ldur    w1, [x1, #31]
b3c:  b801f001    stur    w1, [x0, #31]
b40:  9100afe0    add     x0, sp, #0x2b
b44:  4f000400    movi    v0.4s, #0x0
b48:  3d800000    str     q0, [x0]
b4c:  3c80d000    stur    q0, [x0, #13]
b50:  97fffffa2    bl      9d8 <rand_func>
b54:  910023e0    add     x0, sp, #0x8
b58:  97ffff56    bl      8b0 <puts@plt>

```

根据上面的代码，我们认为#0xc40处是字符串的位置。我们需要验证一下：

```

00000C40  74 6F 75 63 68 20 2F 74 6D 70 2F 4C 61 62 34 5F  t o u c h   / t m p / L a b 4 _
00000C50  74 61 73 6B 33 5F 6E 6F 74 5F 79 6F 75 5F 6E 65  t a s k 3 _ n o t _ y o u _ n e
00000C60  65 64 00 00 00 00 00 00 00 00 00 00 00 00 00 00  e d .
00000C70  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

说明buf确实放在了0xc40处。

根据：

```

18 void file_checker(void) {
19     char filepath[40] = "/tmp/Lab4_task3_23307130447";
20     if (access(filepath, F_OK) != -1)
21         write(STDOUT_FILENO, "Lab4 task3 Success !\n", 21);
22     else
23         write(STDOUT_FILENO, "fail\n", 5);
24 }

```

我们要做的第一件事是把上面0xc40处的字段改成：

```
touch /tmp/Lab4_task3_23307130447
```

由main函数的汇编代码可知，在调用 puts 函数前，对 buf 进行了截取的操作，如果只修改 buf 可能会导致输出不完整。所以需要对调用前的指令进行修改，让修改后的字符串完整地存入栈内，以便正常调用。这也是我们需要做的第二件事，简而言之就是验证，如果不对就还要接着改。

我们要做的第三件事是将puts函数劫持为 system 函数，需要在 main 函数的汇编代码中修改bl指令，使得其调用 system 函数而非 puts 函数。

## 调试以输出完整字符串

```

00000C40  74 6F 75 63 68 20 2F 74 6D 70 2F 4C 61 62 34 5F  t o u c h   / t m p / L a b 4 _
00000C50  74 61 73 6B 33 5F 32 33 33 30 37 31 33 30 34 34  t a s k 3 _ 2 3 3 0 7 1 3 0 4 4
00000C60  37 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  7

```

首先先把buf改掉，然后跑一次看一下：

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_new
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [10:10:51]
$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_new
touch /tmp/Lab4_task3_23307130447
fail
```

可见buf字符串已经完整地显示，下面只需要进行劫持 system 函数操作。

## PLT/GOT表偏移计算过程

首先我们需要看一下system函数和puts函数

```
aarch64-linux-gnu-objdump -D ./task3_new
```

```
00000000000008b0 <puts@plt>:
8b0: f00000f0      adrp    x16, 1f000 <__FRAME_END__+0x1e240>
8b4: f947e211      ldr     x17, [x16, #4032]
8b8: 913f0210      add     x16, x16, #0xfc0
8bc: d61f0220      br     x17
```

```
0000000000000840 <system@plt>:
840: f00000f0      adrp    x16, 1f000 <__FRAME_END__+0x1e240>
844: f947c611      ldr     x17, [x16, #3976]
848: 913e2210      add     x16, x16, #0xf88
84c: d61f0220      br     x17
```

我们的核心代码就是：

```
b58: 97ffff56      b1      8b0 <puts@plt>
```

这个97是跳转的目的地址小于当前PC值；ffff56 为偏移量。

偏移量的计算公式为：**(目的地址-当前地址)/4。结果用补码表示。**

我们先拿puts函数算一下来验证：(0x8b0-0xb58)/4=-170（十进制）->ff56是补码，代表-170，正确。

我们现在算正确的偏移量，也就是拿system来算：(0x840-0xb58)/4=-198(十进制)=补码：0xffff3a

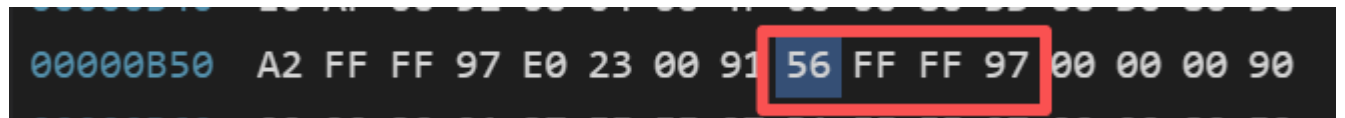
也就是说，我们要把97ffff56改成97ffff3a

## 二进制文件编辑器中修改内容截图

我们的小端法表示如下，我们需要修改b58处的内容：

```
3a ff ff 97
```

原先的截图：



修改后的截图：



## 运行修改后二进制文件截图

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_new
```

```
# pore @ pore in ~/pore26/pore_23307130447/Lab4 on git:master x [10:56:49]
$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_new
Lab4 task3 Success !
```